EWB COURSES

LEARN. ELEVATE. EMPOWER.

CYBERSECURITY MINI PROJECT WORKBOOK

PHISHING URL AWARENESS CHECKER

A complete, step-by-step guide: from an empty folder to a finished project on GitHub

YOUR MISSION

Build a Python program that checks URL text for selected warning signs. Test it, document it, publish it on GitHub.

BUILD • TEST • DOCUMENT • SUBMIT

Language: Python 3.10+ Level: Beginner Time: 3 to 5 days

Final submission: your GitHub repository URL

Certificate eligibility is subject to project review and verification.

Contents

Followthesectionsinorder.Tick every box as you finish a step.

00 Start Here 3
01 Project Brief 4
02 Words You Will Meet 4
03 What You Need and How to Install It 5
04 Create the Project Folder 6
05 Understand the Idea First 7
06 Build It Step by Step 9
07 The Code, Block by Block 12
08 Test Your Project 13
09 Take Your Screenshots 14
10 Make It Your Own 15
11 Write Your README.md 16
12 Publish on GitHub 17
13 Final Check and Submission 19
14 Evaluation Rubric 19
15 Be Ready to Explain Your Work 20
16 Safety and Ethics 20
17 Troubleshooting 21
18 Final Deliverables and Progress Tracker 22

How to use this workbook

Read one section, do it on your computer, then tick it off. Do not skip the testing and README sections: they carry 55 of the 100 marks. If something goes wrong, jump to Section 17 (Troubleshooting).

00 Start Here

Imagine asecurity guard at a building gate. A parcel arrives. The guard does **not** open it. He only looks at the label: strange sender? wrong address? too many stickers? If something looks odd, he says "Review carefully." Your program does the same job for web links. It reads the link **as text** and points out warning signs. It never visits the link.

Your journey in 8 stages

Stage	What you do	Section	Output
1	Understand phishing and URLs	01, 02, 05	You can explain it in your own words
2	Install tools and create the folder	03, 04	Python works, folder is ready
3	Type the code and run it	06, 07	Program prints results
4	Test with five sample URLs	08	tests/test_samples.md
5	Take screenshots	09	Two PNG files
6	Add two improvements	10	Better program
7	Write the README	11	README.md
8	Publish and submit	12, 13	Public GitHub link

Suggested 5-day plan

Day	Goal	Finish line
Day 1	Read Sections 01 to 05. Install Python, VS Code. Create the folder.	Python version works
Day 2	Section 06 and 07. Type the code, run it, understand each block.	First result appears on screen
Day 3	Sections 08 and 09. Five tests, filled table, screenshots.	tests/test_samples.md is complete
Day 4	Sections 10 and 11. Two improvements and the README.	README explains everything
Day 5	Sections 12 and 13. GitHub upload and final check.	Link opens in a private window

Five golden rules

- **Type, run, break, fix.** Making mistakes is how you learn. Read the error message slowly; it usually tells you the line number.
- **Sample strings only.** Use only the URLs given in this workbook. Never test with real suspicious links.
- **Text only.** Your program must never open a URL, scan a website or send data anywhere.
- **Honest results.** Write the real output in your tests, even if a test fails. Then fix it and test again.
- **Your own work.** You may follow the steps, but your improvements and README must be written in your words.

Windows or Mac?
Commands in this workbook are shown for both. On Windows use `python` (or `py`). On Mac and Linux use `python3` wherever you see `python`.

01 Project Brief

Language	Level	Time	Cost
Python 3 (3.10 or later)	Beginner	3 to 5 days	Free. No API keys, no accounts needed except GitHub

Phishing is a trick where a criminal sends a fake link or message to make you share a password, card number or OTP. The fake link often looks like a real one. In this project you build a small command-line tool (a program you run by typing in a terminal window) that inspects the text of a URL for a few visible warning signs.

What your project must do

- Ask the user to type a URL as text.
- Check six warning signs: an **@ symbol**, an **unusually long URL**, an **IP-address hostname**, **many hostname segments**, a **punycode marker** and the **HTTP scheme**.
- Print every warning sign found, plus a cautious message.
- Include tests, a README, screenshots and a public GitHub repository.

What this tool does NOT do
It does not visit websites, scan for malware, ask reputation services or prove a URL is safe. **"No warning signs found" never means "safe".** Say this clearly in your README.

Learning outcomes

- Understand basic phishing risks and the parts of a URL.
- Use Python functions, strings, conditions and regular expressions.
- Run a program from a terminal and check expected behaviour with tests.
- Document and publish a project on GitHub, then explain it with confidence.

02 Words You Will Meet

Keepthispage open. Every difficult word is explained in simple language.

Word	Simple meaning	Example
Phishing	A trick that uses fake links or messages to steal your information.	A fake "bank" message with a link
URL	The full web address of a page. Also called a link.	https://www.example.com/login
Scheme	The start of the URL. It says how to connect: HTTP or HTTPS.	https
Hostname	The name of the website's computer. This is the most important part to read.	www.example.com

Word	Simple meaning	Example
Subdomain	Extra names placed in front of the main domain, separated by dots.	www in www.example.com
Path	Which page inside the website.	/login
IP address	A number address of a computer, like a house number instead of a house name.	192.0.2.10
Punycode	A way to write special (non-English) letters using normal letters. It starts with xn--. Attackers can use look-alike letters.	xn--example
HTTP / HTTPS	HTTPS is the secure, locked version. HTTP is not encrypted. Even HTTPS does not prove a site is honest.	http:// vs https://
Regular expression (regex)	A short pattern used to find text, like a smart "Find" tool.	digits-dot-digits-dot...
Function	A named mini-machine: you give input, it gives output.	inspect_url(...)
Terminal	A text window where you type commands. Command Prompt on Windows, Terminal on Mac.	python --version
Repository (repo)	A project folder stored on GitHub.	phishing-url-awareness
Commit	A saved snapshot of your files with a short message.	"Submit cybersecurity mini project"

03 What You Need and How to Install It

- Computer (Windows, Mac or Linux) and internet access
- Python 3.10 or later
- A code editor: VS Code is recommended
- A free GitHub account
- Basic file and folder skills: create folder, save file, copy and paste

1 Install Python

Windows	Mac
1. Open **python.org** and click Downloads, then the yellow Download Python button. 2. Open the downloaded file. 3. **Tick the box "Add python.exe to PATH"** on the first screen. This is the most important click. 4. Click **Install Now**, wait, then Close.	1. Open **python.org** and download the macOS installer. 2. Open the file and keep clicking Continue, then Install. 3. Enter your Mac password if asked. 4. Always type python3 (not python) in the Terminal.

2 Check that Python works

Open a terminal. **Windows:** press the Windows key, type cmd, press Enter. **Mac:** press Cmd + Space, type Terminal, press Enter. Then type:

WINDOWS

```
python --version

(if that fails, try)
py --version
```

MAC / LINUX

```
python3 --version
```

YOU SHOULD SEE SOMETHING LIKE

```
Python 3.13.1
```

Any version that starts with **3.10 or higher** is fine. If you see an error, go to Troubleshooting (Section 17).

3 Install VS Code (your code editor)

- Open **code.visualstudio.com**, download for your system and install with the default options.
- Open VS Code. Click the Extensions icon (four squares on the left) and search **Python**. Install the one by Microsoft. This gives helpful colours and error hints.

4 Create a GitHub account

- Open **github.com** and click Sign up. Use your email and create a password.
- Choose a **professional username**. It appears in your project link, for example github.com/your-username/phishing-url-awareness.
- Open the verification email and confirm your address. Write your username on the last page of this workbook.

Keep your password private
Never write your GitHub password in a file, a README, a screenshot or a chat message.

04 Create the Project Folder

Your project is a folder with a fixed structure. The reviewer expects exactly these names, so copy them carefully. Use small letters, hyphens in the folder name and underscores in the Python file name.

FINAL STRUCTURE

```
phishing-url-awareness/ ├──
phishing_checker.py     ├──      <- your Python program
README.md                        <- project instruction manual
├── requirements.txt    ├──      <- list of extra packages (we need none)
tests/
│    └── test_samples.md         <- test evidence table
└── screenshots/
     ├── sample_run.png          <- one program run
     └── test_results.png        <- proof of your tests
```

Option A: create it with commands

WINDOWS (COMMAND PROMPT)

```
cd %USERPROFILE%\Documents
mkdir phishing-url-awareness
cd phishing-url-awareness
mkdir tests
mkdir screenshots
type nul > phishing_checker.py
type nul > README.md
type nul > requirements.txt
type nul > tests\test_samples.md
dir
```

MAC / LINUX (TERMINAL)

```
cd ~/Documents
mkdir -p phishing-url-awareness/tests phishing-url-awareness/screenshots
cd phishing-url-awareness
touch phishing_checker.py README.md requirements.txt tests/test_samples.md
ls
```

Option B: create it by clicking

- Create thefolder**phishing-url-awareness** in Documents.
- Open VS Code, choose **File, Open Folder** and select it.
- In the Explorer panel, use the New File and New Folder icons to make each item from the structure above.

Windows trap: hidden file extensions
If Windows hides extensions, your file may secretly be named phishing_checker.py.txt . In File Explorer open the **View** menu and tick **File name extensions** so you can see real names.

Check yourself: type dir (Windows) or ls (Mac). You should see **phishing_checker.py, README.md, requirements.txt, tests** and **screenshots**. The two PNG files are created later in Section 09.

05 Understand the Idea First

A URL is built from parts. Your program focuses on the **hostname**, because that tells you who you are really talking to.



How your program thinks (the flow)

1. Read the text	2. Clean (strip spaces)	3. Add http:// if missing	4. Split into parts	5. Run six checks	6. Print findings

The six warning signs

#	Warning sign	Why people look at it	Honest limit
1	**@ symbol**	Text before @ is ignored by the browser. In example.com@sample.invalid the visit goes to **sample.invalid**, but a quick glance shows example.com.	Rare in normal links, but possible.
2	**Very long URL** (over 100 characters)	Long links can hide the real destination far to the right.	Many honest links are long too.
3	**IP address as hostname**	Real brands use names, not number addresses.	Some internal tools use IPs legitimately.
4	**Many hostname segments** (5 or more parts)	Stacking names like a.b.c.d.e makes the link look official.	Big companies sometimes use many parts.
5	**Punycode marker** (xn--)	Can hide look-alike foreign letters that mimic real names.	Real international sites use it too.
6	**HTTP instead of HTTPS**	HTTP is not encrypted, so data can be seen on the way.	HTTPS alone does not prove trust.

Big lesson
These are **clues**, not proof. A link with zero clues can still be dangerous (for example, a name with a slightly wrong spelling), and a link with clues can be harmless. Good security tools combine many signals. Yours is a learning tool.

06 Build It Step by Step

1 Open your project in VS Code

- Start VS Code. Choose **File, Open Folder** and pick **phishing-url-awareness**.
- Click **phishing_checker.py** in the left panel. A blank editor opens.

2 Add the code

Type the code below into **phishing_checker.py**. Typing (instead of copying) helps you learn, and it avoids hidden-character problems that sometimes appear when copying from a PDF. Python is strict about **indentation** (the empty space at the start of a line). Use **4 spaces** for each level. Do not mix Tab and spaces.

PHISHING_CHECKER.PY

```
import re
from urllib.parse import urlparse


def inspect_url(raw_url):
    """Check visible URL text for selected warning signs."""
    url = raw_url.strip()
    findings = []

    if not url:
        return ["No URL entered."]

    candidate = url if "://" in url else "http://" + url
    parsed = urlparse(candidate)
    host = (parsed.hostname or "").lower()

    if not host:
        return ["Could not identify a hostname."]

    if "@" in candidate:
        findings.append("Contains an @ symbol; inspect carefully.")

    if len(url) > 100:
        findings.append("Unusually long URL (over 100 characters).")

    if re.fullmatch(r"(?:\d{1,3}\.){3}\d{1,3}", host):
        findings.append("Uses an IP address instead of a familiar domain.")

    labels = [part for part in host.split(".") if part]
    if len(labels) >= 5:
        findings.append("Contains many hostname segments/subdomains.")

    if "xn--" in host:
        findings.append("Contains a punycode-style hostname.")

    if parsed.scheme.lower() == "http":
        findings.append("Uses HTTP. HTTPS alone does not prove trust.")

    if not findings:
        findings.append("No listed warning signs found. This does NOT prove safety.")

    return findings


def main():
    print("EWB Phishing URL Awareness Checker")
    print("Text checks only - this program does not open URLs.")
    raw_url = input("Enter a sample URL to inspect: ")

    print("\nResult:")
    for item in inspect_url(raw_url):
        print("- " + item)

    print("\nNever enter passwords from unexpected links.")


if __name__ == "__main__":
    main()
```

Save the file: **Ctrl + S** (Windows) or **Cmd + S** (Mac). A white dot on the file tab means unsaved changes.

3 Add the requirements file

Open **requirements.txt** and type one comment line, then save. A line starting with # is a note that Python tools ignore:

REQUIREMENTS.TXT

```
# No extra packages needed. This project uses only Python's built-in libraries.
```

4 Open the terminal inside VS Code

- Menu: **Terminal, New Terminal**. A panel opens at the bottom.
- Look at the path in the terminal. It must end with **phishing-url-awareness**. If not, type cd and the folder path, or reopen the correct folder.

5 Run the program

WINDOWS

```
python phishing_checker.py
```

MAC / LINUX

```
python3 phishing_checker.py
```

When it asks for a URL, type this sample and press Enter:

TYPE THIS

```
http://192.0.2.10/account
```

EXPECTED RESULT

```
EWB Phishing URL Awareness Checker
Text checks only - this program does not open URLs.
Enter a sample URL to inspect: http://192.0.2.10/account

Result:
- Uses an IP address instead of a familiar domain.
- Uses HTTP. HTTPS alone does not prove trust.

Never enter passwords from unexpected links.
```

Run it again and try https://www.example.com/login . You should see: No listed warning signs found. This does NOT prove safety.

Checkpoint
If you see the results above, your program works. Take a break, then read Section 07 so you understand every block.

Program closes instantly?
Do not double-click the .py file. Always run it from an open terminal, as shown above, so the window stays open.

07 The Code, Block by Block

You must be able to explain your own code. Read each block and its plain-English meaning.

Block	Code idea	Plain English
1	`import re` `from urllib.parse import urlparse`	Borrow two ready-made tools. **re** finds text patterns. **urlparse** cuts a URL into parts (like a knife that separates scheme, hostname and path).
2	`def inspect_url(raw_url):`	Create a named mini-machine. Input: the text typed by the user. Output: a list of messages.
3	`url = raw_url.strip()` `findings = []`	**strip()** removes extra spaces at both ends. **findings** is an empty notebook where we write each warning.
4	`if not url: return [...]`	If nothing was typed, stop early with a friendly message.
5	`candidate = ... "http://" + url`	If the user forgot the start (http:// or https://), add one so the cutting tool can find the hostname. **Note:** this is why `www.example.com` typed without a scheme shows the HTTP warning.
6	`parsed = urlparse(candidate)` `host = ...lower()`	Cut the URL into parts and keep the hostname in small letters, so comparing is easy.
7	`if "@" in candidate`	Check 1. Is there an @ anywhere in the link?
8	`len(url) > 100`	Check 2. Count the characters. More than 100 is flagged.
9	`re.fullmatch(r"(?:\d{1,3}\.){3}\d{1,3}", host)`	Check 3. Does the hostname look like **number.number.number.number**? `\d{1,3}` means 1 to 3 digits, `\.` means a real dot, `{3}` means repeat three times.
10	`labels = host.split(".")` `len(labels) >= 5`	Check 4. Split the hostname at each dot into pieces and count them. `a.b.c.d.e.example.invalid` has 7 pieces.
11	`"xn--" in host`	Check 5. Punycode names start their pieces with xn--.
12	`parsed.scheme.lower() == "http"`	Check 6. Is the connection type plain HTTP?
13	`if not findings:`	If the notebook is still empty, say no listed signs were found, and remind that this is **not** proof of safety.
14	`def main():`	The "front desk": prints the title, asks for input, prints each finding with a dash, then a safety reminder.
15	`if __name__ == "__main__":`	Means: *run main() only when this file is started directly.* If another file imports your code later, it will not run by accident.

Try it yourself

Change the number 100 to 40, save, run again with a long sample and watch the result change. Then change it back. Playing with one small value is the fastest way to understand code.

08 Test Your Project

Testingmeans checking that the program gives the result you **expected**. You will run five sample URLs, write down what you expected, what you actually got, and whether it passed. Use **only** these strings. They use reserved example-style domains and documentation IP space, so they are safe.

#	Sample input (type exactly)	Expected observation	What you should actually see
1	https://www.example.com/login	May show no listed warning signs; not a safety guarantee.	No listed warning signs found. This does NOT prove safety.
2	http://192.0.2.10/account	Flags IP-address hostname and HTTP.	Uses an IP address instead of a familiar domain. Uses HTTP. HTTPS alone does not prove trust.
3	http://example.com@sample.invalid/login	Flags @ symbol and HTTP.	Contains an @ symbol; inspect carefully. Uses HTTP. HTTPS alone does not prove trust.
4	https://a.b.c.d.e.example.invalid/path	Flags many hostname segments.	Contains many hostname segments/subdomains.
5	https://xn--example.invalid/	Flags punycode-style hostname.	Contains a punycode-style hostname.

Reference results were produced by running the exact code in Section 06 with Python 3.13. Your wording must match because you typed the same messages. If it differs, compare your code carefully.

1 Fill the test file

Open **tests/test_samples.md** and paste this table. Replace the blank cells with what **your** screen shows, and write PASS or FAIL.

TESTS/TEST_SAMPLES.MD

```
# Test Results

Python version used: 3.__          |      Operating system: _______          |      Date: _______

| # | Input | Expected result | Actual result | Pass/Fail | |---|-------|-----------------|---------------|----
-------|
| 1 | https://www.example.com/login | No listed warning signs | | |
| 2 | http://192.0.2.10/account | IP address + HTTP | | |
| 3 | http://example.com@sample.invalid/login | @ symbol + HTTP | | |
| 4 | https://a.b.c.d.e.example.invalid/path | Many hostname segments | | | | 5 | https://xn--
example.invalid/ | Punycode-style hostname | | |

## Notes
Write 2 or 3 lines: what you learned, and what surprised you.
```

2 Run each test

- Run python phishing_checker.py once per sample. Type the sample, press Enter, read the result.
- Copy the printed warning lines into the Actual column (short form is fine).
- Mark **PASS** if the actual result matches the expected observation. If not, mark **FAIL**, fix your code, run again and update the row honestly.

Bonus checks (optional, great for your Notes)

Try typing	What happens	What it teaches
(just press Enter)	No URL entered.	The program handles empty input.
www.example.com	Uses HTTP. HTTPS alone does not prove trust.	Missing scheme is treated as http://
https://secure-login.example.invalid	No listed warning signs found.	A tricky name can slip through. No signs does not mean safe!
http://[bad	The program stops with ValueError: Invalid IPv6 URL	A real gap. You will fix it in Section 10 (Improvement C).

09 Take Your Screenshots

Screenshots are proof that **your** program ran on **your** computer. You need two images in the **screenshots** folder.

File name	What it must show
sample_run.png	The terminal with the command you typed, the question, one sample URL and the printed result.
test_results.png	Evidence of all five tests: either your filled **tests/test_samples.md** opened in VS Code, or the terminal showing the five runs.

Windows	Mac
Press **Windows + Shift + S**, drag a box around the area, open **Paint**, press Ctrl + V, then File, Save as **PNG**.	Press **Cmd + Shift + 4**, drag a box around the area. The image lands on your Desktop. Rename it and move it into the screenshots folder.

- The terminal path and the command are visible.
- Text is big enough to read (zoom in the terminal with Ctrl + Plus / Cmd + Plus).
- No passwords, personal emails, tokens or private tabs are visible.
- The file names are exactly sample_run.png and test_results.png, saved inside **screenshots/**.

10 Make It Your Own

Oncethe starter works, add **at least two improvements** and explain them in your README. Pick from the menu. Examples below are for learning: study them, then write **your own version** in your own style.

Option	Improvement	Level
A	Loop: check several URLs without restarting, with a quit command	Easy
B	Add a clear "Review carefully" summary when warnings are found	Easy
C	Handle malformed input clearly, so the program never crashes	Medium
D	Explain each warning in simple words (why it matters)	Medium
E	Improve output formatting (numbers, spacing, a simple menu)	Easy

Not allowed
No live scanning, no opening URLs, no internet requests and no website probing. Keep it local, safe and easy to explain.

A Loop with quit (replaces main)

A **while True** loop repeats forever until we **break** out of it. Typing q ends the program.

EXAMPLE: IMPROVED MAIN()

```
def main():
    print("EWB Phishing URL Awareness Checker")
    print("Text checks only - this program does not open URLs.")

    while True:
        raw_url = input("\nEnter a sample URL (or type q to quit): ")

        if raw_url.strip().lower() in ("q", "quit", "exit"):
            print("Goodbye! Stay alert online.")
            break

        print("\nResult:")
        for item in inspect_url(raw_url):
            print("- " + item)

        print("Never enter passwords from unexpected links.")
```

B Review carefully summary

First add a small helper that answers: are these real warnings, or just information messages? Then use it inside the loop.

EXAMPLE: HELPER FUNCTION (PLACE ABOVE MAIN)

```
INFO_MESSAGES = ("No URL entered", "Could not identify", "No listed warning")

def has_warnings(findings):
    return not findings[0].startswith(INFO_MESSAGES)
```

EXAMPLE: USE IT INSIDE MAIN()

```
    findings = inspect_url(raw_url)
    foritem in findings:
        print("- " + item)

    if has_warnings(findings):
        print("\nREVIEW CAREFULLY: " + str(len(findings)) + " warning sign(s) found.")
```

WHAT YOU WILL SEE FOR HTTP://192.0.2.10/ACCOUNT

```
Result:
- Uses an IP address instead of a familiar domain.
- Uses HTTP. HTTPS alone does not prove trust.

REVIEW CAREFULLY: 2 warning sign(s) found.
```

C Stop crashes on malformed input

Problem: typing http://[bad makes Python raise a **ValueError** and the program stops. Fix: wrap the risky line in **try / except**. It means: try this; if it fails, do the safe thing instead. In inspect_url, replace the line parsed = urlparse(candidate) with:

EXAMPLE: SAFER PARSING

```
try:
    parsed = urlparse(candidate)
except ValueError:
    return ["This text is not a valid URL format."]
```

D Explain each warning (hint)

Create a Python **dictionary** (a list of key and value pairs) that maps each warning text to a one-line reason, then print the reason under the warning. Example idea: REASONS = {"@": "The browser ignores text before @."}. Design the rest yourself.

After every change, test again
Re-run all five tests from Section 08. If an old test now fails, your improvement broke something. Fix it before moving on. In your README, say what you improved, why, and how you checked it.

11 Write Your README.md

The README is the front page of your project. A stranger should understand it in two minutes. Paste this template into **README.md** and replace every line in [square brackets] with your own words.

README.MD TEMPLATE

```
# EWB Phishing URL Awareness Checker

## Overview
[2-3 sentences: what phishing is, and what your tool does.]

## Features
- Detects an @ symbol in the URL
- Flags unusually long URLs (over 100 characters)
- Flags IP-address hostnames
- Flags many hostname segments (5 or more)
- Flags punycode-style hostnames (xn--)
- Flags the HTTP scheme
```

```
- [Your improvement 1] -
[Your improvement 2]

## Requirements
Python 3.10 or later. No extra packages.

## How to Run
Windows:     python phishing_checker.py
Mac/Linux: python3 phishing_checker.py

## Example Output
[Paste the output of one run here.]

## Testing
| # | Input | Expected | Actual | Result |
|---|-------|----------|--------|--------|
| 1 | https://www.example.com/login | No warning signs | [fill] | [PASS] |
[Add rows 2 to 5. Full table is in tests/test_samples.md]

## Screenshots
![Sample run](screenshots/sample_run.png)
![Test results](screenshots/test_results.png)

## Improvements
1. [Name] - [what changed, why, how you tested it] 2.
[Name] - [what changed, why, how you tested it]

## Limitations
This is an educational tool. It only reads URL text. It does not visit
URLs, scan for malware or prove that a URL is safe. No warning signs
found does NOT mean a URL is safe.

## Ethical Use
Use sample strings only. Do not collect credentials. Do not use this
project to deceive anyone or to test websites without permission.

## Author
[Your name], [batch or course], EWB Courses
```

Make the screenshots appear

The two lines starting with ![...] show your images on the GitHub page. The folder name and file names must match exactly, including small letters.

12 Publish on GitHub

GitHubis a website that stores your project online so the reviewer can open it. Choose **Method A** (easiest) or **Method B** (the professional way).

Method A: upload in the browser

1 Create the repository

- Sign in at github.com. Click the **+** icon at the top right, then **New repository**.
- Repository name: phishing-url-awareness
- Description: Python tool that checks URL text for phishing warning signs (educational project)
- Choose **Public** so the reviewer can open it (unless your coordinator tells you otherwise).
- Leave "Add a README" **unticked**, because you already wrote one. Click **Create repository**.

2 Upload your files

- On the empty repository page, click **uploading an existing file** (or Add file, then Upload files).
- Open your project folder on your computer. Select **phishing_checker.py, README.md, requirements.txt** and the folders **tests** and **screenshots**. Drag them all into the browser box. Dragging the folders keeps the structure.
- Wait until every file shows a green tick. Scroll down.
- Commit message: Submit cybersecurity mini project. Click **Commit changes**.

3 Check your repository

- You should see all five items in the file list and your README displayed underneath.
- Click **tests** and **screenshots** and confirm the files are inside.
- Open README.md on GitHub and check that both screenshots appear.

4 Test the link in a private window

- Copy the address from the browser bar. It looks like https://github.com/YOUR-USERNAME/phishing-url-awareness.
- Open a private window: **Ctrl + Shift + N** (Chrome, Edge), **Ctrl + Shift + P** (Firefox), **Cmd + Shift + N** (Mac Chrome).
- Paste the link. If your project opens without signing in, the reviewer can open it too.

Method B (optional): upload with Git

Git is a tool that saves versions of your project and sends them to GitHub. Install it from **git-scm.com**, then close and reopen your terminal and check git --version. Do the one-time setup:

ONE-TIME SETUP (USE YOUR OWN DETAILS)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Create the empty repository on GitHub first (Method A, Step 1). Then, **inside your project folder**:

UPLOAD COMMANDS

```
git init
git add phishing_checker.py README.md requirements.txt tests screenshots
git commit -m "Add cybersecurity mini project"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/phishing-url-awareness.git
git push -u origin main
```

- Replace **YOUR-USERNAME** with your real GitHub username and copy the exact repository URL.
- A browser window may open asking you to sign in to GitHub. Do that and return to the terminal. GitHub does not accept your normal account password for Git commands.
- If you ever need a token, keep it private. **Never put passwords or tokens in source files.**

Changing something later?
Method A: open the file on GitHub, click the pencil icon, edit and commit. Method B: change the file, then run git add ., git commit -m "Update README" and git push.

13 Final Check and Submission

Tickeachbox. Do not submit until all are ticked.

- **Program:** runs without errors from a terminal and accepts input.
- **Detection:** all six checks work and you can explain each one.
- **Tests:** five sample tests have input, expected result, actual result and pass/fail.
- **Improvements:** at least two are implemented and explained in the README.
- **README:** includes overview, features, requirements, how to run, testing, screenshots, limitations, ethical use and improvements.
- **Screenshots:** show the real program and results, with readable text and nothing private.
- **GitHub:** repository is Public, files are visible, and the link opens in a private window.
- **Clean:** no passwords, API keys, private data or unrelated files (also no __pycache__ folders).
- **Understanding:** you can explain what the program does and what it cannot do (Section 15).

How to submit
Submit the **accessible GitHub repository URL** using the internship team's designated submission method. A screenshot alone is not a substitute for the repository link.

Certificate
Certificate issuance is subject to successful project submission and verification. A link alone does not guarantee a certificate.

14 Evaluation Rubric

Criteria	Marks	Reviewer looks for	How to earn full marks
Working program	25	Runs and accepts input.	No errors; handles empty input; run from a terminal.
Detection logic	20	Checks are implemented and explained.	All six checks work; you can explain each in simple words.
Testing and evidence	20	Five tests and screenshots.	Filled table with honest results; two clear screenshots.
README and documentation	20	Clear setup, usage, limitations and ethics.	Every README section is complete and written by you.
GitHub quality and explanation	15	Accessible repo; student understands work.	Public repo, neat structure, and confident answers in Section 15.
TOTAL	**100**	Completeness and understanding.	

15 Be Ready to Explain Your Work

Areviewer may ask you questions about your project. Practise saying these answers in your own words.

Question	Model answer (simple)
What is phishing?	A trick where fake links or messages try to make people share passwords or payment details.
Does your tool open the URL?	No. It only reads the text of the URL. Nothing is visited or sent anywhere.
What is a hostname?	The name of the website's computer, such as www.example.com. It tells you who you are really connecting to.
Why is an @ symbol a warning?	The browser ignores the text before @, so a link can look like one site but go to another.
Why is an IP address suspicious?	Honest services normally use readable names. A bare number hides who owns the site.
What is punycode?	A way to store non-English letters using normal letters, starting with xn--. Attackers can use look-alike letters.
Is a URL safe if no warning appears?	No. My tool checks only six clues. A dangerous link can have none of them.
Whatdoes if__name__== "__main__" do?	It runs main() only when the file is started directly, not when another file imports it.
Which improvements did you make?	Name two, say why you chose them, and say how you tested that nothing broke.
What would you add next, safely?	A check for suspicious words in the path, a simple score, or more explanations, all still working on text only.

16 Safety and Ethics

- **Defensive learning only.** Do not send phishing messages or collect credentials.
- Do not scan or probe websites or networks without written authorisation.
- Do not open suspicious links to test them. Use only the sample strings in this workbook.
- Never upload passwords, API keys or confidential data to GitHub.
- Do not claim this tool can definitively classify URLs as safe or malicious.
- Submit your own work. Learning from this guide is encouraged; copying someone else's project is not.

If you upload a secret by mistake
Revoke or rotate it immediately (change the password or token), then follow GitHub's history-cleanup guidance. Deleting the file alone is not enough, because old versions stay in history.

17 Troubleshooting

Problem	What to try
python is not recognised	Try `py --version` on Windows. If it still fails, reinstall Python and tick **Add python.exe to PATH**, then open a new terminal.
Mac says `command not found: python`	Use `python3` instead.
Program closes immediately	Run it from an open terminal in the project folder, not by double-clicking.
`can't open file ... No such file`	Your terminal is in the wrong folder. Use cd to enter **phishing-url-awareness** and check with dir / ls.
File is named `.py.txt`	Show file extensions (View, File name extensions), then rename to `phishing_checker.py`.
`IndentationError` or `SyntaxError`	Check indentation: 4 spaces per level, no Tabs mixed in. Look at the line number in the error. Retyping the block usually fixes PDF-copy problems.
`NameError: name ... is not defined`	A name is misspelled or a function is missing. Compare spelling and capital letters with the code.
`ValueError: Invalid IPv6 URL`	You typed something like `http://[bad`. Fix it with Improvement C in Section 10.
Strange symbols in the output	Use a plain hyphen (-) in your print messages, as in this workbook, instead of fancy dashes.
No warning appears	Normal. Some inputs have none of the six signs. That does not prove safety.
Screenshot is blurry or too small	Zoom the terminal text, capture only the needed area and save as PNG.
GitHub drag-and-drop skips folders	Use Chrome or Edge, or click **choose your files**, or use Git (Method B).
`git` is not recognised	Install Git from git-scm.com, then close and reopen the terminal.
`remote origin already exists`	Run `git remote set-url origin YOUR-REPOSITORY-URL` and push again.
Git asks for a password and refuses	Sign in through the browser window Git opens. Normal account passwords are not accepted for Git commands.
Reviewer cannot open the link	Make the repository Public (Settings, scroll to Danger Zone, Change visibility) or grant reviewer access. Test in a private window.
Link shows 404 Not Found	Check the username and repository spelling, and that the repository is not private.

.

18 Final Deliverables and Progress Tracker

Deliverable	Where	Done
phishing_checker.py: source code	Project root	☑
README.md: documentation	Project root	☑
requirements.txt	Project root	☑
tests/test_samples.md: testing evidence	tests/	☑
sample_run.png and test_results.png	screenshots/	☑
GitHub repository URL: final submission	Submitted as instructed	☑

My details

Name	ANKITA AJAY BORADE
Batch / Course	**BATCH 1 , Ewb AI POWERED CYBERSECURITY INTERNSHIP**
GitHub username	**AnkitaaBorade**
Repository URL	https://github.com/AnkitaaBorade/phishing-url-awareness _
Submission date	10 - 11 - 2026

BUILD WITH CARE. TEST HONESTLY. DOCUMENT CLEARLY. SUBMIT YOUR OWN WORK.
EWB EDUTECH PRIVATE LIMITED | Learn. Elevate. Empower.